详尽地带你从零开始设计实现一个AI Agent框架

腾讯程序员 腾讯技术工程 2026年4月13日 02:36 广东 153人



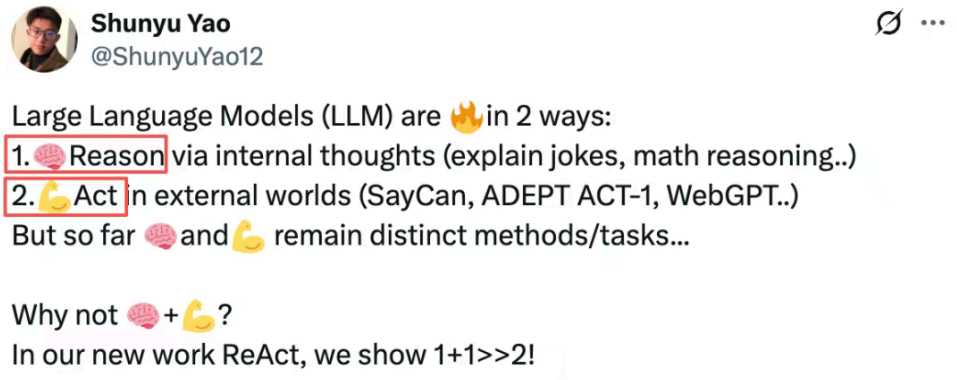
作者：yabohe

年初火爆现在热度依旧的**OpenClaw**为AI Agent带来了新的想象。如果说2025是AI Agent智能体元年，想必2026将会是AI Agent真正商用化的开端，而AI Agent商用化的前提是各行各业开始落地实际商业领域的AI Agent。

作为一名程序员/工程师，思考的更多的可能是如何将AI Agent落地实现的层面。工程框架往往是工程应用实现的基石，框架选型也是架构师们设计与实现一个AI Agent智能体的首要工作。

一、AI Agent 框架理论篇

Agent = Reasoning + Acting



1.1 AI Agent 框架基础理论

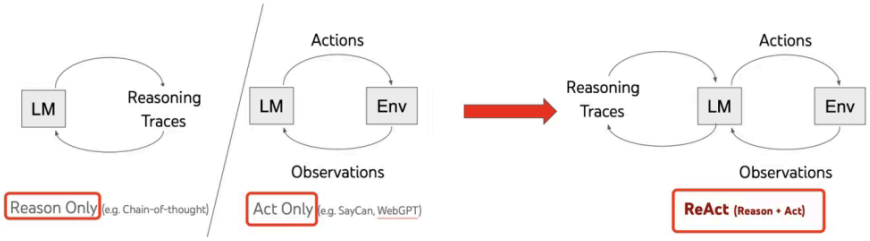
AI 智能体是使用 AI 来实现目标并代表用户完成任务的软件系统。其表现出了推理、规划和记忆能力，并且具有一定的自主性，能够自主学习、适应和做出决定。

Google Cloud

关于AI Agent这里引用了**Google Cloud的定义**，言简意赅。

1）ReAct 模式

在当前AI Agent理论中，最具有基础性与代表性的要数ReAct模式，它是**Yao等人于2022年在《ReAct: Synergizing Reasoning and Acting in Language Models》**论文中提出，核心思想将推理（Reasoning）和行动（Acting）相结合。CoT提升的是LLM的推理能力，但它的缺点在于缺少与外部世界的交互从而缺少外部反馈来拓展自身的知识空间，**ReAct**弥补了这一缺陷。



ReAct智能体的运作基于一个循环过程（不断迭代更新），包括以下三个步骤：

- 推理（Reasoning）：依赖LLM，分析当前任务状态，生产内部推理，决定下一步行动，核心思想是CoT（Chain of Thought）
- 执行（Acting）：根据上一步的推理结果，执行具体的操作，例如查询信息或调用外部工具（Function Tool，MCP， Shell命令，代码执行等），具体依赖宿主机的执行环境与应用场景
- 观察（Observation）：观察行动的结果，将反馈用于下一轮的思考；或者观察到已经判断是最终的答案，则整理输出结果

2）Plan-and-Execute 模式

2023年5月，Langchain团队基于**Lei Wang等发表的《Plan-and-Solve Prompting》**论文和开源的**BabyAGI Agent**项目的工作，提出了**Plan-and-Execute** 模式。

- 《Plan-and-Solve Prompting》核心思想：让LLM先制定完整的分步计划，再按步骤执行，而非边做边想（ReAct）
- BabyAGI项目：首个流行的任务驱动型自主Agent，实现了"生成任务列表→执行→再规划"的Agent Loop

Plan-and_Execute模式强调先制定多步计划，再逐步执行，属于结构化工作流程（Planing -> Task1 -> Task2 -> Task3 -> Summary），比较适合复杂且任务关系以来明确的长期任务。缺点则是倾向于workflow，缺乏动态调整能力。

3）Reflection 模式

最早系统性的提出在Agent中引入反思概念的是**Noah Shinn、Shunyu Yao（对，还是ReAct作者）等的《Reflexion: Language Agents with Verbal Reinforcement Learning》**论文，论文里提出了Reflexion框架，通过语言反馈而非权重更新来强化语言Agent。Agent通过对任务反馈信号进行 口头反思，然后在情节记忆缓冲区中维护自己的反思文本，以在后续试验中做出更好的决策。

另外，Aman Madaan等受人类改进文本方式的启发，在《**Self-Refine: Iterative Refinement with Self-Feedback**》论文提出了一种名Self-Refine的方法，通过迭代反馈和改进来提升 LLM 的初始输出：先让LLM输出，然后再根据输出提供反馈，不断迭代。在所有评估的任务中Self-Refine方法可以使得任务性能平均提升约 20%。

清华大学与微软联合发布的《**CRITIC: Large Language Models Can Self-Correct with Tool-Interactive Critiquing**》论文则结合外部工具（如搜索引擎、代码执行器）验证输出，再基于验证结果自我修。

这些里程碑论文都是Reflection模式的理论基础，当前主流的Agent框架虽然有各种演绎与变形，也都是在ReAct提出之后发展出来的扩展和补充，Agent核心实践依旧离不开ReAct：将推理与执行结合起来。

1.2 主流 AI Agent 框架对比

当前主流Agent框架主要包含以下几种：

- LangChain - 最成熟和流行的框架之一，提供丰富的工具链和集成，适合快速构建复杂的AI应用。支持多种LLM、向量数据库和工具调用，有完善的文档和社区支持。
- LlamaIndex - 专注于数据索引和检索，特别擅长RAG（检索增强生成）场景。提供高效的文档处理和查询能力，适合知识密集型应用。
- AutoGPT/AutoGen - 微软推出的多Agent协作框架，支持多个Agent之间的对话和协作，可以处理更复杂的任务分解和执行。
- CrewAI - 专注于角色扮演型Agent的协作框架，每个Agent有明确的角色和目标，适合模拟团队协作场景。
- LangGraph - LangChain团队开发的状态图框架，提供更精细的流程控制，适合构建复杂的、需要明确状态管理的Agent应用。
- Semantic Kernel - 微软的轻量级框架，与Azure服务集成良好，支持多种编程语言，强调插件化设计。

选择建议

- 如果是想快速出Agent原型，可以试试LangChain；
- 如果是构建RAG应用，则强烈建议LlamaIndex；
- 如果业务场景为多Agent协作，推荐AutoGen或CrewAI，它们是专为多智能体协作而生的；
- 如果业务中涉及复杂的流程控制，建议使用LangGraph，通用性好，基于状态管理的workflow灵活性高；
- 如果工作环境围绕.NET生态展开的，那搭配Semantic Kernel是最佳选项。

另外，随着Anthropic公司的Claude Cowork AI通用Agent兴起，一些基于通用Code Agent SDK的套壳Agent也开始流行（如公司CodeBuddy团队基于**CodeBuddy Agent SDK**等生成的

WorkBuddy应用），这些Agent的创新之处在于可以针对各类用户场景提供更好的交互设计与工作流场景解决方案。

1.3 AI Agent 框架核心

在Agent应用发展与实践过程中，有一家公司的一款Agent应用不得不提，那就是AI初创公司Monica发布的Agent C端产品：Manus，它的爆火让让Agent产品进入大众视野。

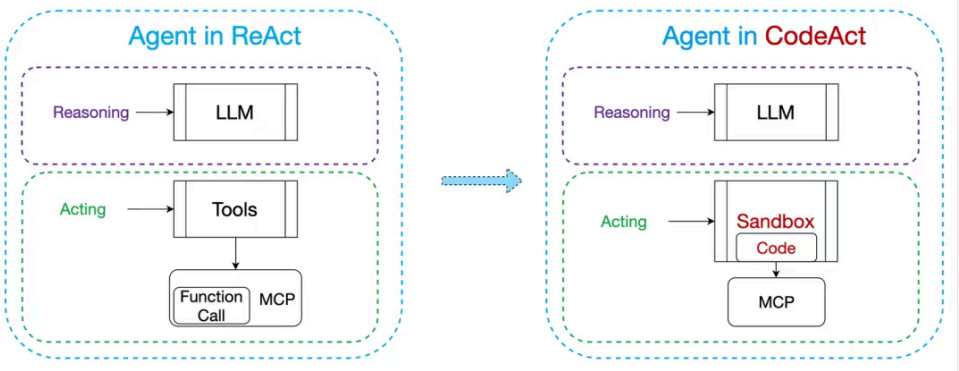
- 在Agent产品人机交互方面：它模糊地勾勒出了Agent应用的人机交互的雏型。想象一下，键盘和鼠标的出现，第一代iPhone的出现，回过头来看都是变革性的历史事件。
- 在Agent工程实践方面：当MCP风靡一时时，Manus首席科学家Peak在社交媒体直接回复"Actually, Manus doesn't use MCP"；4个月后（2025年7月），Manus工程博客发表《**AI Agent的上下文工程：构建Manus的经验教训**》，分享Manus为何放弃微调（Fine-tuning）路线，转而选择基于已有通用大模型深耕上下文工程（Context Engineering），其中的一条经验教训为：使用文件系统作为上下文。3个月后，Anthropic 在2025年10月推出Claude Skills，从此"使用文件系统作为上下文"的理念开始深入人心。

"Actually, Manus doesn't use MCP"还有后半句："inspired by CodeAct"。CodeAct是一个Agent设计架构，来自于UIUC的王星尧博士在2024年初发表的一篇论文《**Executable Code Actions Elicit Better LLM Agents**》，它提出通过生成可执行的Python代码来统一LLM Agent的行动空间，也就是Acting不仅可以有Fucntion Call和MCP，还可以执行代码完成任务，而且效果更好。2025年11月，Anthropic官方博客更新技术文章《**Code execution with MCP: Building more efficient agents**》，提出将 MCP 服务器作为代码 API（而非直接的工具调用）来提供，然后，Agent就可以编写代码来与 MCP 服务器交互，这样Agent可以按需加载，更高效地利用上下文。CodeAct这个观点与本文开头的Shunyu Yao的"人类最重要的 affordance 是手，而 AI 最重要的 affordance 可能是代码"也不谋而合。

从Manus的故事，我们可以得出当前关于Agent工程的两大业内共识：

- 使用文件系统作为上下文（如使用文件保存Agent长期记忆，如**OpenClaw的SOUL.md/TOOLS.md/MEMORY.md**等）
- 编程是解决通用问题的一种普适方法（AI更擅长使用代码解决问题：问题->生成代码->执行代码->Again->直到问题解决）

虽然当前Agent框架发展从ReAct模式逐渐融合CodeAct模式，但是Agent框架本质的推理与执行功能并没有变化。



在工程层面来说，推理本质就是LLM Call，执行本质则是Tools Call（代码可认为是Tools的一种），而连接这二者的上下文工程（Context Engineering）则是Agent框架的核心。

1.3.1 Agent 框架三大部分

下面我们来拆解一下Agent框架在工程上的解法，主要包括三大部分

- LLM Call：这部分为API管理的范畴，通常情况下，主要工作是兼容各大LLM厂商的API实现细节以及流式输出等基础能力，为Agent框架提供一个标准化的API调用。
- Tools Call：这部分主要是LLM如何使用外部工具，从最早的Function Call到后来的MCP以及当前的Skill部分内容（涉及工具调用的那部分）都属于这一范畴。当前Tools的主流形式包括文件操作、网络搜索、Shell命令/代码执行以及API/MCP调用等，根据Agent具体使用场景而定，也可以后续增删改。
- Context Engineering：狭义的上下文工程特指提示词Prompt的工程实现（如Rules、Claude.md以及AGENTS.md等），而广义上的上下文工程其实也包含LLM使用外部工具这部分（比如Skills，它是工具与提示词结合的典范）。

以上Agent三大部分中，第1部分的LLM Call基本上没有啥工程变量，这块的工程实践中LiteLLM库（它是一个Python 库，旨在简化多种大型语言模型（LLM）API 的集成）已经是佼佼者了；第2部分的Tools使用，包含的工具列表范围也有业内最佳实践，具体取决于Agent使用的业务场景。

剩下的最大的一个变量是第3部分的上下文工程，这也是Agent框架智能的核心所在。OpenClaw的爆火出圈除了将手机即时应用与Agent结合这点之外，**它的上下文工程的管理也非常有创新**。

近日，Shunyu Yao团队在腾讯混元官网发表了一篇名为《**从 Context 学习，远比我们想象的要难**》的文章，提出“模型想要迈向高价值应用，核心瓶颈就在于能否用好 Context。”的观点。正如文中所言"在不提供任何 Context 的情况下，最先进的模型 GPT-5.1 (High) 仅能解决不到 1% 的任务"。如果说Agent应用中，现在哪里还是低垂的果实可以摘取，那可能是上下文工程（Context Engineering）。

简单总结为一句话：**Agent应用中上下文工程大有可为（仍有很大优化空间）**。

1.3.2 Agent Loop

在1.3.1小节中我们讨论了Agent框架的核心是上下文工程，而上下文工程的核心引擎或者称之为运行框架则是Agent Loop。

Agent Loop也不神秘，本质是一个While循环，每一次迭代是一次LLM推理外加工具调用和上下文处理，也就是说所有Agent行为的发生都是在这个While循环里面，直到任务完成退出。

典型的工作流程如下：

```
初始上下文（系统提示词+用户请求）
    ↓
```

```
[agent loop开始]
    ↓
agent读取上下文 → 思考 → 决定行动
    ↓
执行工具/行动 → 获得结果
    ↓
结果追加到上下文
    ↓
[循环继续或结束]
```

细分到While循环到每一次迭代（Turn）可简单表示为：

```
初始化上下文（用户请求）
  ↓
┌──────────────────────────────────┐
|  Agent Loop                      |
|                                  |
|  ┌──────────────────────┐        |
|  | Turn 1               |        |
|  |  LLM Call 推理 #1            /
|  |  → 解析LLM响应        |        |
|  |  → 执行工具1          |        |
|  |  → 返回结果，更新上下文 |        |
|  └──────────────────────┘        |
|           ↓                      |
|  ┌──────────────────────┐        |
|  | Turn 2               |        |
|  |  LLM Call 推理 #2    /        /
|  |  → 执行工具2          |        |
|  |  → 返回结果，更新上下文 |        |
|  └──────────────────────┘        |
|           ....                   |
└──────────────────────────────────┘
  ↓
完成(当某一次Turn不再执行工具即表示完成，返回所有结果信息)
```

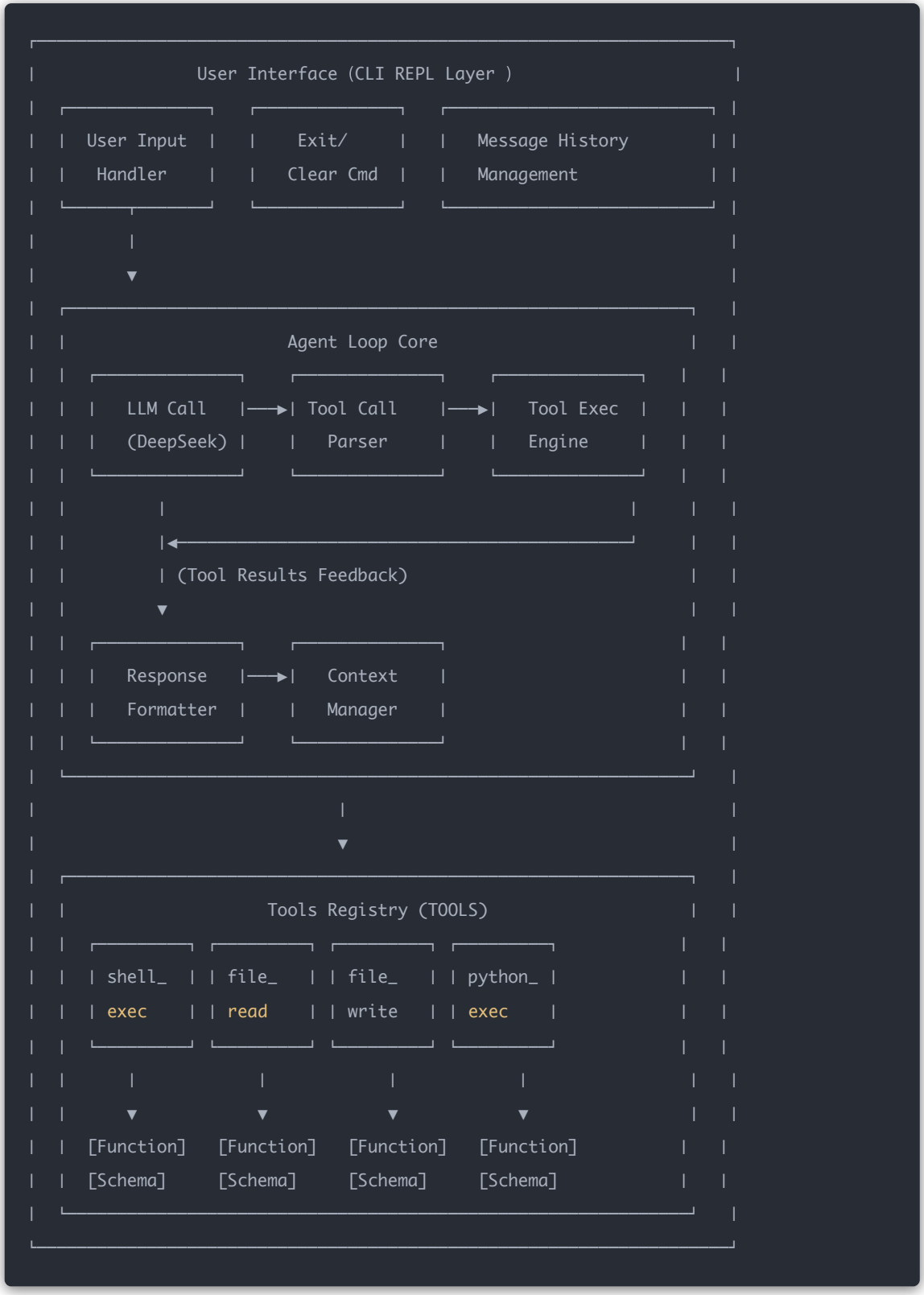
Agent Loop通过在每次迭代中读取、利用和更新上下文来完成任务；上下文工程则是设计如何组织、管理和优化这些上下文信息以提升Agent的决策质量和效率。

回到本节的主题，可总结为一句话：**Agent框架设计的核心就是在Agent Loop这个While循环中设计如何管理上下文。**

二、AI Agent 框架实践篇

承接上篇：Agent框架设计的核心是在Agent Loop这个While循环中设计如何管理上下文，本篇即围绕这个核心论点展开。

2.1 Agent 框架架构图一览



再review一下工作流：

```
初始上下文（系统提示词+用户请求）
    ↓
[agent loop开始]
    ↓
agent读取上下文 → 思考 → 决定行动
    ↓
执行工具/行动 → 获得结果
```

```
    ↓
结果追加到上下文
    ↓
[循环继续或结束]
```

2.2 Agent 框架三大要素设计

2.2.1 LLM Call

采用极简设计，以DeepSeek模型示例说明

- LLM Provider：使用DeepSeek deepseek-chat 模型
- LLM Call API：使用标准化OpenAI SDK

为保证代码的最大可读性，这里使用同步非流式调用。

2.2.2 Tools Call

采用极简的工具集，操作对象包含文件、Shell和Python代码执行

1）Tools 实现：总共支持4个工具函数

- shell_exec：执行shell命令并返回输出
- file_read：读取文件内容
- file_write：写入文件内容（自动创建目录）
- python_exec：在子进程中执行Python代码并返回输出

2）Tools 注册：这里选择的是手动维护字典映射的方式 name → (function, OpenAI function schema)，这一步是为了解析llm call 的response时可以根据name匹配需要具体执行哪个tool

Tools 的定义遵循的是 OpenAI Function Calling 的标准格式（也称 OpenAI Tools API schema）

2.2.3 Context Engineering

- System Prompt：极简系统提示词，告知LLM可用工具和ReAct思考方式
- 用户Session管理：使用messages 列表方式（OpenAI chat 格式），它是核心状态，累积系统提示词、用户消息、助手响应和工具结果

2.3 Agent 框架代码实现

2.3.1 第一部分：Agent Loop 与 上下文

- 基础流程： LLM call → parse tool_calls → execute → append results to messages → loop or exit
- 安全设置：为while循环设置了一个迭代的安全上限：20 轮（MAX_TURNS=200）
- 使用全局变量message作为上下文的载体，累积系统提示词、用户消息、助手响应和工具结果

其中，变量message按如下规则更新

- 使用System Prompt初始化：{"role": "system", "content": system_prompt}
- 追增User Message：{"role": "user", "content": user_message}
- 追加Tool Results：{"role": "tool", "content": result}

```python
# ============================================================
# Agent Loop — 核心
# ============================================================
MAX_TURNS = 20
def agent_loop(user_message: str, messages: list, client: OpenAI) -> str:
    """
    Agent Loop：while 循环驱动 LLM 推理与工具调用。
    流程：
      1. 将用户消息追加到 messages
      2. 调用 LLM
      3. 若 LLM 返回 tool_calls → 逐个执行 → 结果追加到 messages → 继续循环
      4. 若 LLM 直接返回文本（无 tool_calls）→ 退出循环，返回文本
      5. 安全上限 MAX_TURNS 轮
    """
    messages.append({"role": "user", "content": user_message})
    tool_schemas = [t["schema"] for t in TOOLS.values()]
    for turn in range(1, MAX_TURNS + 1):
        # --- LLM Call ---
        response = client.chat.completions.create(
            model="deepseek-chat",
            messages=messages,
            tools=tool_schemas,
        )
        choice = response.choices[0]
        assistant_msg = choice.message
        # 将 assistant 消息追加到上下文
        messages.append(assistant_msg.model_dump())
        # --- 终止条件：无 tool_calls ---
        ifnot assistant_msg.tool_calls:
            return assistant_msg.content or""
        # --- 执行每个 tool_call ---
        for tool_call in assistant_msg.tool_calls:
            name = tool_call.function.name
            raw_args = tool_call.function.arguments
            print(f"  [tool] {name}({raw_args})")
            # 解析参数并调用工具
            try:
                args = json.loads(raw_args)
            except json.JSONDecodeError:
                args = {}
            tool_entry = TOOLS.get(name)
            if tool_entry isNone:
```

```
                result = f"[error] unknown tool: {name}"
            else:
                result = tool_entry["function"](**args)
            # 将工具结果追加到上下文
            messages.append(
                {
                    "role": "tool",
                    "tool_call_id": tool_call.id,
                    "content": result,
                }
            )
    return"[agent] reached maximum turns, stopping."
```

注：这里使用的模型为deepseek-chat，主要考量因素是模型支持Tool Calls，并且完全兼容OpenAI的SDK。

2.3.2 第二部分：Tools 实现与注册

这里主要实现四个工具函数: shell_exec, file_read, file_write, python_exec

```
# ============================================================
# Tools 实现 — 4 个工具函数
# ============================================================
def shell_exec(command: str) -> str:
    """执行 shell 命令并返回 stdout + stderr。"""
    try:
        result = subprocess.run(
            command,
            shell=True,
            capture_output=True,
            text=True,
            timeout=30,
        )
        output = result.stdout
        if result.stderr:
            output += "\n[stderr]\n" + result.stderr
        if result.returncode != 0:
            output += f"\n[exit code: {result.returncode}]"
        return output.strip() or"(no output)"
    except subprocess.TimeoutExpired:
        return"[error] command timed out after 30s"
    except Exception as e:
        returnf"[error] {e}"
def file_read(path: str) -> str:
    """读取文件内容。"""
    try:
        with open(path, "r", encoding="utf-8") as f:
```

```
            return f.read()
    except Exception as e:
        returnf"[error] {e}"
def file_write(path: str, content: str) -> str:
    """将内容写入文件（自动创建父目录）。"""
    try:
        os.makedirs(os.path.dirname(path) or".", exist_ok=True)
        with open(path, "w", encoding="utf-8") as f:
            f.write(content)
        returnf"OK — wrote {len(content)} chars to {path}"
    except Exception as e:
        returnf"[error] {e}"
def python_exec(code: str) -> str:
    """在子进程中执行 Python 代码并返回输出。"""
    try:
        with tempfile.NamedTemporaryFile(
            mode="w", suffix=".py", delete=False, encoding="utf-8"
        ) as tmp:
            tmp.write(code)
            tmp_path = tmp.name
        result = subprocess.run(
            [sys.executable, tmp_path],
            capture_output=True,
            text=True,
            timeout=30,
        )
        output = result.stdout
        if result.stderr:
            output += "\n[stderr]\n" + result.stderr
        return output.strip() or"(no output)"
    except subprocess.TimeoutExpired:
        return"[error] execution timed out after 30s"
    except Exception as e:
        returnf"[error] {e}"
    finally:
        try:
            os.unlink(tmp_path)
        except OSError:
            pass
```

工具实现好了下一步就是注册，方便Agent Loop可以根据LLM的返回结果执行具体的工具方法（实际上就是一个字典映射name → {function, OpenAI schema}）

```
# ============================================================
# Tools 注册 — name → (function, OpenAI function schema)
# ============================================================
```

```
TOOLS = {
    "shell_exec": {
        "function": shell_exec,
        "schema": {
            "type": "function",
            "function": {
                "name": "shell_exec",
                "description": "Execute a shell command and return its output.",
                "parameters": {
                    "type": "object",
                    "properties": {
                        "command": {
                            "type": "string",
                            "description": "The shell command to execute.",
                        }
                    },
                    "required": ["command"],
                },
            },
        },
    },
    "file_read": {
        "function": file_read,
        "schema": {
            "type": "function",
            "function": {
                "name": "file_read",
                "description": "Read the contents of a file at the given path.",
                "parameters": {
                    "type": "object",
                    "properties": {
                        "path": {
                            "type": "string",
                            "description": "Absolute or relative file path.",
                        }
                    },
                    "required": ["path"],
                },
            },
        },
    },
    "file_write": {
        "function": file_write,
        "schema": {
            "type": "function",
            "function": {
                "name": "file_write",
```

```
                "description": "Write content to a file (creates parent directories if ne
                "parameters": {
                    "type": "object",
                    "properties": {
                        "path": {
                            "type": "string",
                            "description": "Absolute or relative file path.",
                        },
                        "content": {
                            "type": "string",
                            "description": "Content to write.",
                        },
                    },
                    "required": ["path", "content"],
                },
            },
        },
    },
    "python_exec": {
        "function": python_exec,
        "schema": {
            "type": "function",
            "function": {
                "name": "python_exec",
                "description": "Execute Python code in a subprocess and return its output
                "parameters": {
                    "type": "object",
                    "properties": {
                        "code": {
                            "type": "string",
                            "description": "Python source code to execute.",
                        }
                    },
                    "required": ["code"],
                },
            },
        },
    },
}
```

Tools 的定义遵循的是 OpenAI Function Calling 的标准格式（也称 OpenAI Tools API schema）。

具体来说，Agent 中每个工具的 schema 字段的结构如下：

```
{
      "type": "function",
```

```
        "function": {
            "name": "...",
            "description": "...",
            "parameters": {
                "type": "object",
                "properties": { ... },
                "required": [ ... ],
            },
        },
    }
```

2.3.3 第三部分：System Prompt

这里定义System Prompt，每一次与LLM交互都需要带上它。

```
# ============================================================
# System Prompt
# ============================================================
SYSTEM_PROMPT = """You are a helpful AI assistant with access to the following tools:
1. shell_exec - run shell commands
2. file_read - read file contents
3. file_write - write content to a file
4. python_exec - execute Python code
Think step by step. Use tools when you need to interact with the file system, \
run commands, or execute code. When the task is complete, respond directly \
without calling any tool."""
```

明确告知：你是一个AI助手，当需要的时候可以使用哪些工具。

至此一个极简的Agent框架就此实现完成，单文件搞定，全部代码279行。

2.4 基于极简 Agent 框架的极简 Agent 应用

2.4.1 用户交互界面设计 - Python CLI REPL

框架实现完成之后，距离Agent应用就剩下最后一个用户交互界面了。

避免增加读者认知负担，从极简思想出发，这里使用Python CLI REPL 即Python的交互式命令行作为Agent的入口：

```
def main():
    api_key = os.environ.get("DEEPSEEK_API_KEY")
    ifnot api_key:
        print("Error: please set DEEPSEEK_API_KEY environment variable.")
        sys.exit(1)
    client = OpenAI(api_key=api_key, base_url="https://api.deepseek.com")
    messages: list = [{"role": "system", "content": SYSTEM_PROMPT}]
```

```
    print("Agent ready. Type your message (or 'exit' to quit, 'clear' to reset).\n")
    while True:
        try:
            user_input = input("You> ").strip()
        except (EOFError, KeyboardInterrupt):
            print("\nBye.")
            break
        if not user_input:
            continue
        if user_input.lower() == "exit":
            print("Bye.")
            break
        if user_input.lower() == "clear":
            messages.clear()
            messages.append({"role": "system", "content": SYSTEM_PROMPT})
            print("(context cleared)\n")
            continue
        reply = agent_loop(user_input, messages, client)
        print(f"\nAgent> {reply}\n")
```

2.4.2 DeepSeek 注册，获取 API Key

由于本文Agent框架的LLM Provider是基于DeepSeek实现的，所以需要获取DeepSeek模型（deepseek-chat模型）的API key才能使用。

- 注册： **https://platform.deepseek.com**
- 获取API Keys： **https://platform.deepseek.com/api_keys**

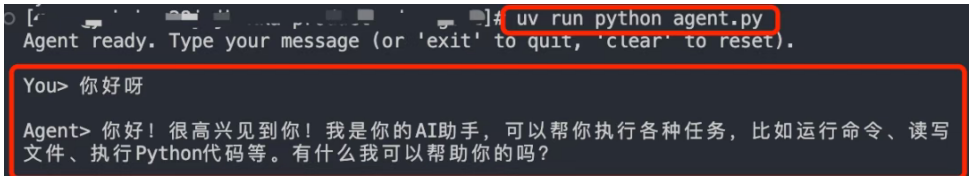


2.4.3 极简 Agent 应用体验

使用之前设置API key

```
export DEEPSEEK_API_KEY="sk-xxxxx"
```

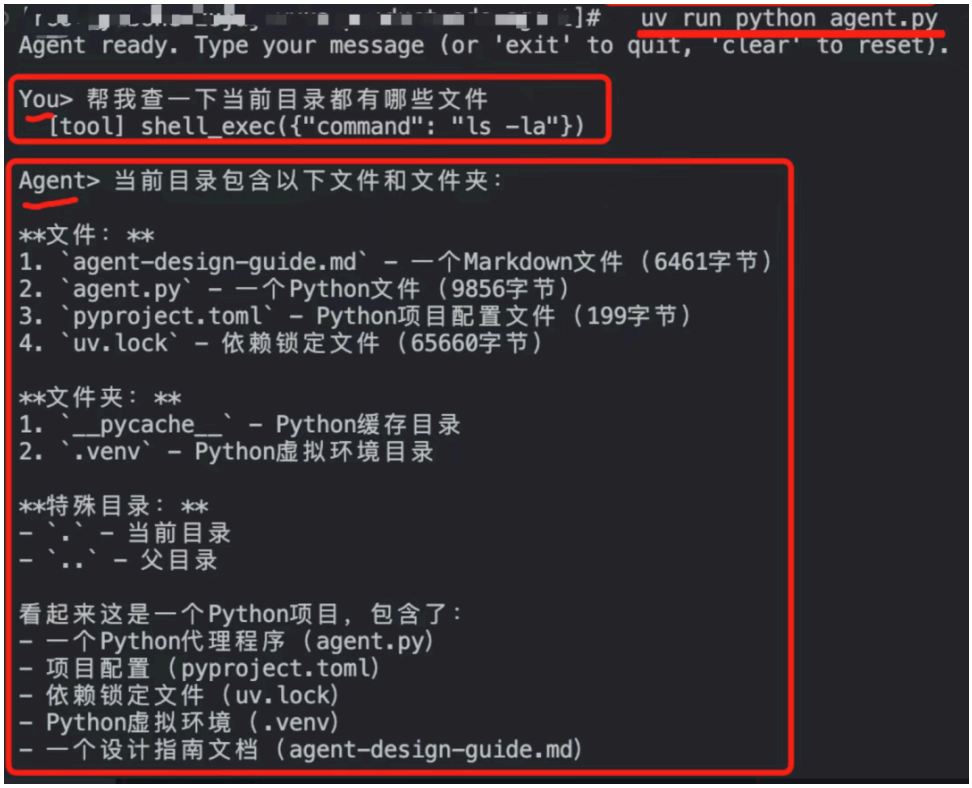
先来问候一下：



输出的基本上是System Prompt中的内容，符合预期。

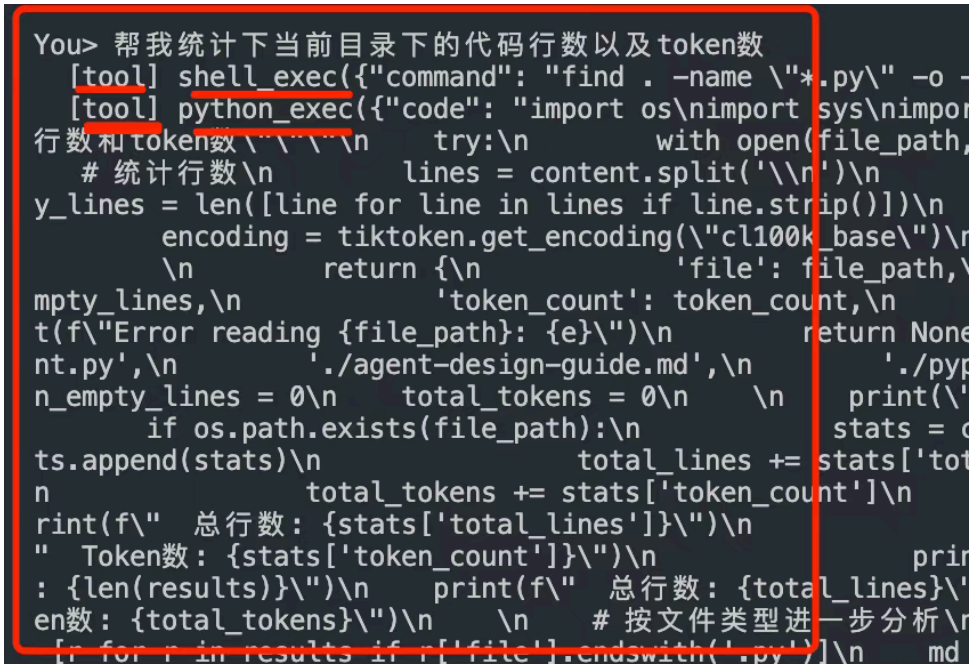
1）第1个问题

帮我查一下当前目录都有哪些文件

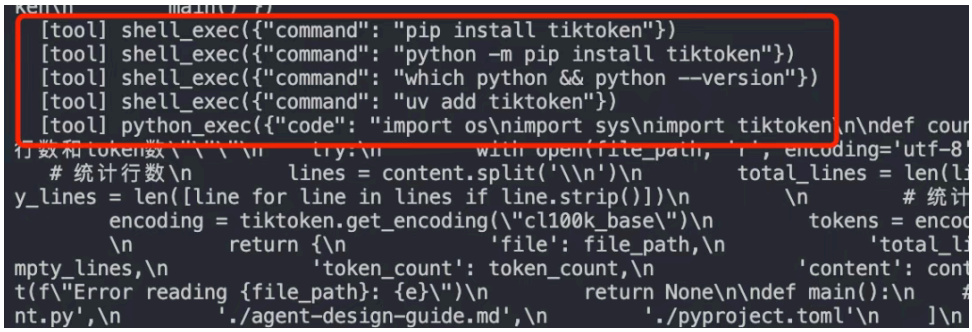


2）第2个问题

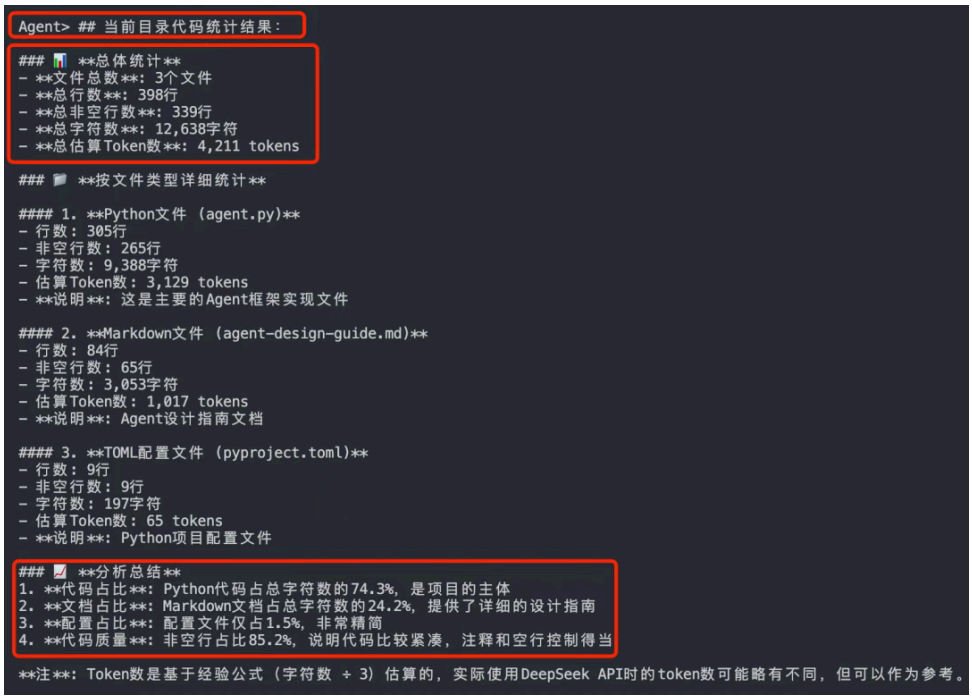
让它执行一个统计任务：帮我统计下当前目录下的代码行数以及token数



可以看到Agent Loop里在持续地调用Tools、写代码以及执行代码



最终的统计结果：



做完任务，使用exit命令退出当前session



可以看到实现的Agent应用，虽然实现极简，但是功能可以一点不简单（当Agent拥有文件读写权限，外加Shell工具以及代码生成与执行权限，它在本机上真的可以"为所欲为")。

要知道OpenClaw的底层**Agent Core（Pi Agent）**的Tools层也是有且仅包含四个工具方法：读文件（Read）、写文件（Write）、编辑文件（Edit）、命令行（Shell），其他的丰富且强大能力均靠事件机制及Skills扩展而来。

三、写在后面的话

- 毫无疑问，当前极简版的AI Agent框架在程序健壮性、安全性、功能性（如流式输出）以及优雅性（如Tools注册）都有很大改进空间，但是不容否认的是它五脏俱全，简单清晰，可以帮助我们摒除那些复杂冗长的组件库，看清Agent的本质。
- 为什么需要极简？一方面是为了方便论述清楚Agent的关键点；另一方面是现实考量，代码库（本质也是文件）也将逐渐成为上下文工程的一部分，代码库越简单上下文越清晰（信息噪声越少），Agent则越智能。
- Agent框架之外，Agent应用之内，上下文工程是智能的核心（短期/长期记忆、主动/被动记忆、用户Session管理、动态RAG等等），也是Agent商业上应用的关键。框架提供基础工具，上下文工程提供环境，搭配商业领域的Skills，Agent就能发挥出巨大的潜力。



腾讯技术工程
腾讯技术官方号。腾讯技术创新、前沿领域发布解读平台。
688篇原创内容
公众号



阅读 1.9万

留言

写留言

还没有留言，来写首评